Week 13 (Semaphore cont.)

✅ Monitor

- condition variable

✅ Java synchronization

- How to use synchronize to solve the problem of bounded buffer

Content

✅ Monitor

(Semaphore is low-level, programmer have to manage Signal and wait by themselves, I fyou did something wrong, there is no compilation error. For this problem, Some OS provide you more high-level mechanism or you may have high-level library to do this without busy waiting)

- and one of them is monitor

The idea of monitor is that it use the concept of OOP.

- when you start oop, what are the major benefit
  - encapsulation → you combine the data and the code that work with data in 1 module (use the idea of information hiding) like data can only be access by this

Monitors

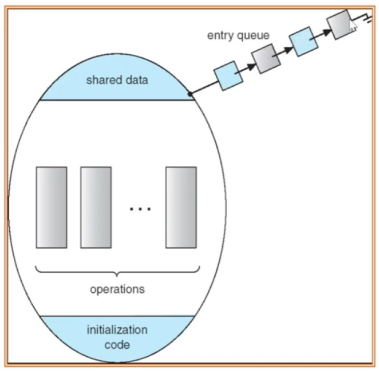
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Only one process may be active within the monitor at a time

```
monitor monitor-name
{
   // shared variable declarations
   procedure P1 (…) { …. }
            …

   procedure Pn (…) {……}

    Initialization code ( ….) { … }
            …
   }
}
```

- you group share variable and function that work with that variable in the same place, and then os gonna control this for you, that only 1 thread or process can work in monitor(active in monitor, one at a time)

(pic 3.45)

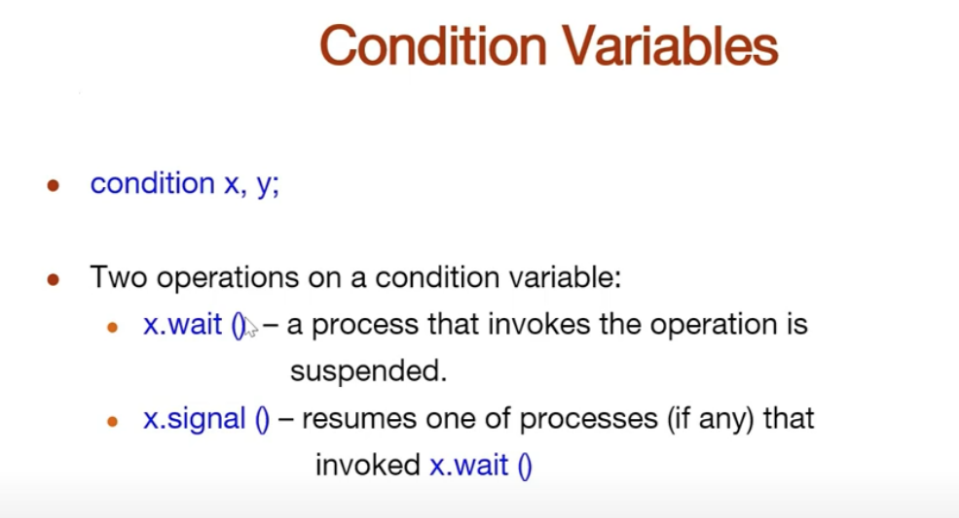
Schematic view of a Monitor



- if there process in here, the rest must wait in this queue
- entry queue is linked list (idea)

- so actually, there may be situation that the process or thread that work in this monitor have to wait for osme condition before it can continue
    - but at that point, it can release the share data without problem, and wait for some event to occur

📍Condition Variables



(To prevent busy waiting → like if it go back to ready queue, and os allow it to work → and it said i still have to wait (this is idea of busy waiting)

- if the thread need that condition → it gonna call wait and this wait is not the same as semaphore → but it wait of the monitor
    - think as wait queue inside monitor
    - the thread or process will call wait it has to wait for some event and it make sure that it allow the resource to be access.
    - the queue outside you can think it as ready queue.
    - the queue inside you can think it as waiting state, waiting queue. (if process move here, the memory will be released for other)
        - the thread that finish using share data can call signal to invoke 1 thread(waiting) and the thread that is invoke have to go back to the end of entry queue and wait for the next turn to go into monitor again (waiting → ready)

    - when that event occur → go back to ready queue

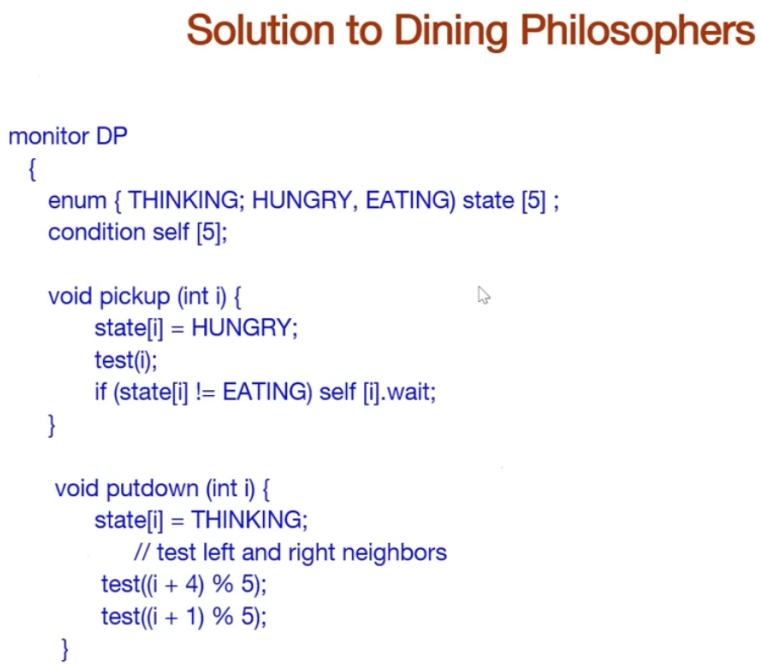
This is high-level, you just declare monitor, the rest is OS task

- for wait and signal here, if the porcess finish, it can call the signal to wake up 1 thread that wait in the condition.

The different between semaphore and monitor is

- signal with a wait doesn't cause any damaage → the process can call signal even when no process is waiting in waiting queue → and there is no problem.
- like ask mom to wake you up, but you already wake up → no problem.
- it will cause big problem if you call signal without wait in semaphore

📍Example of using monitor to solve dining philosophers problem



- we don't want 2 thread or process to grap the same chopsticks at the same time.

- state is state of each philosopher
- each has 3 state (thinking, hungry, eating)

- condition of 5 philosopher

2 function

1. pickup → work with state
2. put down → work with state

function work with share variable group in moniror

Solution to Dining Philosophers (cont)

```
void test (int i) {
        if ( (state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
            state[i] = EATING ;
             self[i].signal () ;
         }
    }

    initialization_code() {
        for (int i = 0; i < 5; i++)
        state[i] = THINKING;
    }
}
```

- before start, each philosopher is in the state of thinking.

Solution to Dining Philosophers (cont)

- Each philosopher *i* invokes the operations pickup() and putdown() in the following sequence:

```
dp.pickup (i)

    EAT

dp.putdown (i)
```

- each philosopher call pickup then eat and putdown

- what gonna happen when you call pickup
  - when you call pickup → the state change to hangry → and it call test to test whether it cna eat or not
    - if after call wait → the state is not eating → it put itself to wait.
  - only 1 process at a time can use pickup, putdown?
  - for putdown → state change to thinking → call test to tell left and right hand that i stop eating, (my state state i is thinking)
    - signal to tell.
- how test work
  - check state of left handside philosopher whether this one is eating or not
  - i am hungry?
  - the right is not eating
  - then, it change its state to eaiting
    - and then call signal → if before that is wait.

Do you think we still have deadlock problem → no

- Deadlock → 1 philosopher pick single chopstick in hand

Starvation → if not deadlock, can still cause starvation

- why we pay attention to deadlock more, because when we talk about deadlock → it mean it can happen to more than 1 process, so if we not solve,..
- starvation → one process

✅ Java synchronization

(java has their own buld in synchronize)

synchronized Statement

- Every object has a lock associated with it
- Calling a synchronized method requires “owning” the lock
- If a calling thread does not own the lock (another thread already owns it), the calling thread is placed in the wait set for the object’s lock
- The lock is released when a thread exits the synchronized method

- Java has this built in synchrnize for all class and object, it dependwhether you want to use it or not
- when you use synchronize → lock it
- if you lock an object, another thread can’t use that object
- good thing → you don’t have to do it explicitly, just use the keyword synchronized

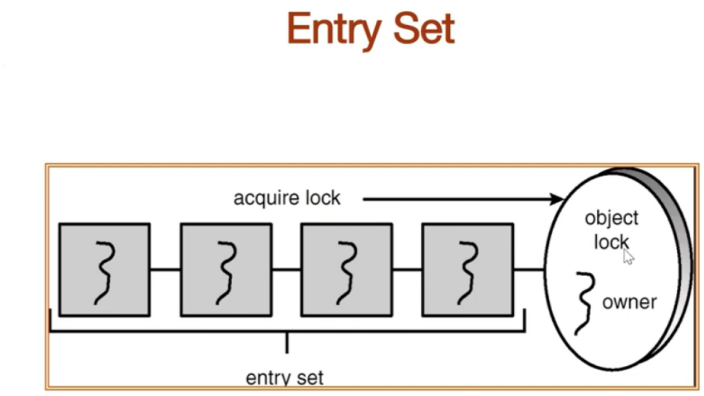
insert() with wait/notify Methods

```
public synchronized void insert(Object item) {
    while (count == BUFFER SIZE) {
                try {
                        wait();
                }
                catch (InterruptedException e) { }
    }
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER SIZE;
    notify();
}
```

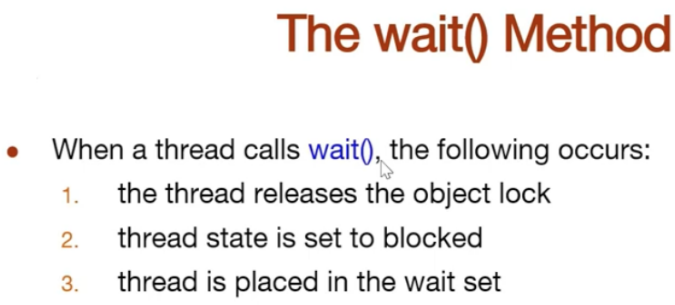
- then when a thread call this method, a object that contain this method is lock

- so this mean, another thread can not call any synchrnized method → this lock occur automatically (when thread call synchronize function)
- when it last execute last line → automatically release resource

The similarity between synchronize and monitor



- because when a thread call synchronzie method, then it seem like that thread is inside, and if other thread want to call this function too, have to wait (entry set, entry queue)
- and java synchronziation also provide condition



- after the thread call wait → put in wait set.

The notify() Method

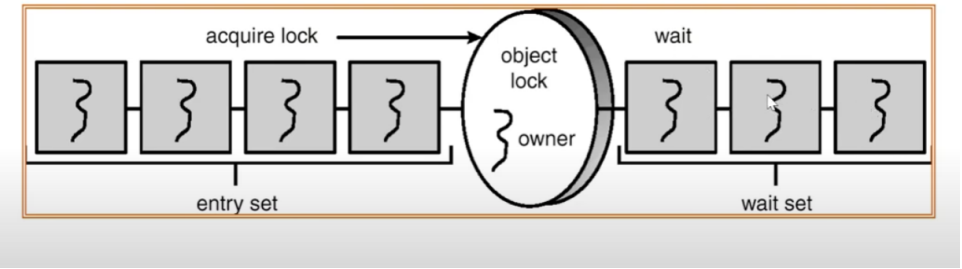
When a thread calls notify(), the following occurs:

1. selects an arbitrary thread *T* from the wait set
2. moves *T* to the entry set
3. sets *T* to Runnable

T can now compete for the object's lock again

- to wake up a thread → java has method name "notify"
  - same as signal (monitor)
- the thread call notiyf to wakt up the thread in wait set

Entry and Wait Sets



- how notify work depend on the implementation of jvm → when you call notify it depend which thread will be wake up (can't choose, jvm choose 1 thread in wait set and put in entry set).
- bounded waiting?

if you think this not fair → use notifyAll() → it may cause bounded waiting

Multiple Notifications

- notify() selects an arbitrary thread from the wait set.
  *This may not be the thread that you want to be selected.
- Java does not allow you to specify the thread to be selected
- notifyAll() removes ALL threads from the wait set and places them in the entry set. This allows the threads to decide among themselves who should proceed next.
- notifyAll() is a conservative strategy that works best when multiple threads may be in the wait set

- notifyAll() → you wake up all the thread in wait set and put it in entry set.

📍How to use synchronize to solve the problem of bounded buffer

- insert call by producer
- remove call by customer

insert() with wait/notify Methods

```
public synchronized void insert(Object item) {
    while (count == BUFFER SIZE) {
                try {
                        wait();
                }
                catch (InterruptedException e) { }
    }
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER SIZE;
    notify();
}
```

- insert method is a method of bounded buffer class, and you put the word synchronize

if there still slot avaible, pass count == BUFER SIZE

- it can make sure that it put data and notify
- if the slot is full → wait

remove() with wait/notify Methods

```
public synchronized Object remove() {
    Object item;
    while (count == 0) {
                try {
                        wait();
                }
                catch (InterruptedException e) { }
    }
    --count;
    item = buffer[out];
    out = (out + 1) % BUFFER SIZE;
    notify();
    return item;
}
```

if count is not 0

- it can read, remove?and notify

now we can use synchronize to solve bounded buffer problem

What is the problem of using synchrnize at a function(method) level?

- using synchronize might reduce the deeply of concurrency (number of thread that can work at a same time)

Synchronize block

insert() with wait/notify Methods

```
public synchronized void insert(Object item) {
    while (count == BUFFER SIZE) {
                try {
                        wait();
                }
                catch (InterruptedException e) { }
    }
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER SIZE;
    notify();
}
```

- if we put it there, it like in your funciton there might be some part that can work concurrenry.
  - this can increase the degree of concurrency in the program
- if you synchronize at function level → i not alow concurrenry.

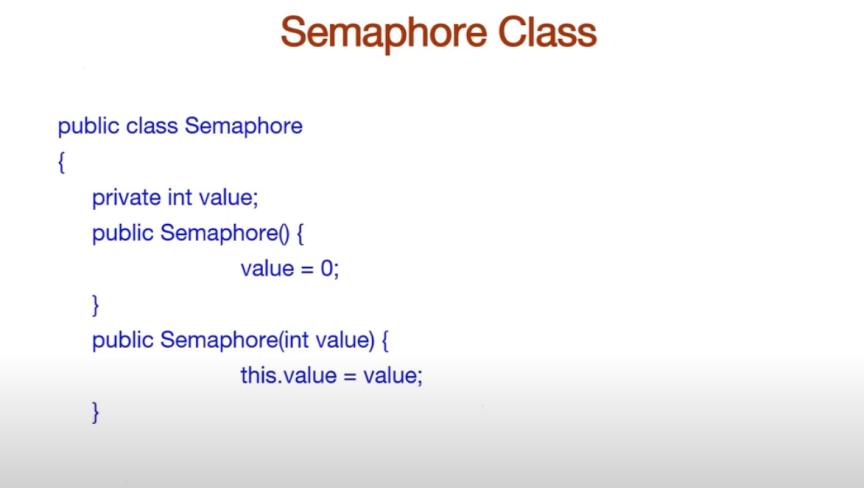
📍Yield

insert() with wait/notify Methods

```
public synchronized void insert(Object item) {
    while (count == BUFFER SIZE) {
                try {
                        wait();
                }
                catch (InterruptedException e) { }
    }
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER SIZE;
    notify();
}
```

- yield → give way to some other car
  - i happy to stop working and go to wait, allow other thread to run.
- in java, if 2 thread has the same priority → java may use first come first serve
- we use yield for co-op programming if we know that the os use non-premptive algo (like fifo)
- if we don't use yield here, the first one that get chance to run may run forever(for loop)
- don't use yield inside synchronize method
  - resource with me but you can run
  - deadlock

✅ Java semaphore



- java has higher level than semaphore
- but java dev want to use semaphore
- java library provide semaphore class
- for java semapore, mehtod acquire is wait, release is signal

Semaphore Class (cont)

```
public synchronized void acquire() {
        while (value == 0)
            try {
                wait();
            } catch (InterruptedException ie) { }
        value--;
    }
    public synchronized void release() {
        ++value;
        notify();
    }
}
```

- it used synchronized keyword
- this use higher level synchronization to implement low-level synchrnization

Quiz

1. The main idea of monitor is to combined shared data and functions to work with shared data into one unit and OS will ensure that one process will be active within a monitor at a time.
    - True
2. The process that is active in a monitor, will be moved back to the entry queue when it needs to wait for some condition to occur
    - False
3. Applying monitor to solve dining philosopher problem using the algorithm in this course may cause the problem of _____.
    - starvation
4. Java provides a mechanism that resembles the idea of monitor is ___
    - synchronization
5. Java synchronization will be occured only the method level.

- False

6. Java synchronization is done by calling lock() and released() method.
   - False (kindof auto?)
7. What is the correct wayt o say that the method is a Java synchrpnized method
   - public synchronized void func()
8. When a thread calls an object synchronized method, the thread will be put to the wait set if there is a therad that previously called the same or different synchronized method and does not finish executing that method.
   - False
9. Calling the yield() method will cause the object to be unlocked to allow opther threads to work with synchronized mehthods
   - False (yield doesn't released the lock)
10. If wait() is called inside java synchronized method, thread will move to the entry set.
    - False (it move to wait)
11. A programmer can use tmethod notify() to choose a specified process from the wait set.
    - False
12. What is the output?

```
import java.util.Random;

//this class counter is share data
public class Counter {
    private int count;
    public Counter(int val) {
        count = val;
    }
    public synchronized void increment()throws InterruptedExcept
```

```
        int temp = count;
        Random rand = new Random();
        int num = rand.nextInt(5);
        Thread.sleep(num*1000);
        temp++;
        count = temp;

    }
    public synchronized void decrement() throws InterruptedExce
        int temp = count;
        Random rand = new Random();
        int num = rand.nextInt(5);
        Thread.sleep(num*1000);
         temp--;
         count = temp;


    }
    public int getCount() {
        return count;
    }
}

public class Decrementer extends Thread {
    public Counter obj;
    public Decrementer(Counter c) {
        obj = c;
    }
    @Override
    public void run() {
        try {
            obj.decrement();
        } catch (InterruptedException ex) {

        }
    }
```

```
}
public class Incrementer extends Thread {
    public Counter obj;
    public Incrementer(Counter c) {
        obj = c;
    }
    @Override
    public void run() {
        try {
            obj.increment();
        } catch (InterruptedException ex) {

        }
    }
}

public class Main {
    public static void main (String[] args) throws InterruptedEx
        Counter counter = new Counter(8);
        Incrementer incThread = new Incrementer(counter);
        Decrementer decThread = new Decrementer(counter);
        incThread.start();
        decThread.start();
        incThread.join();
        decThread.join();
        System.out.println(counter.getCount());
    }
}
```

- Answer is 8 → synchronize method (each thread is also execute only 1)

13. What (is)are the possible output?

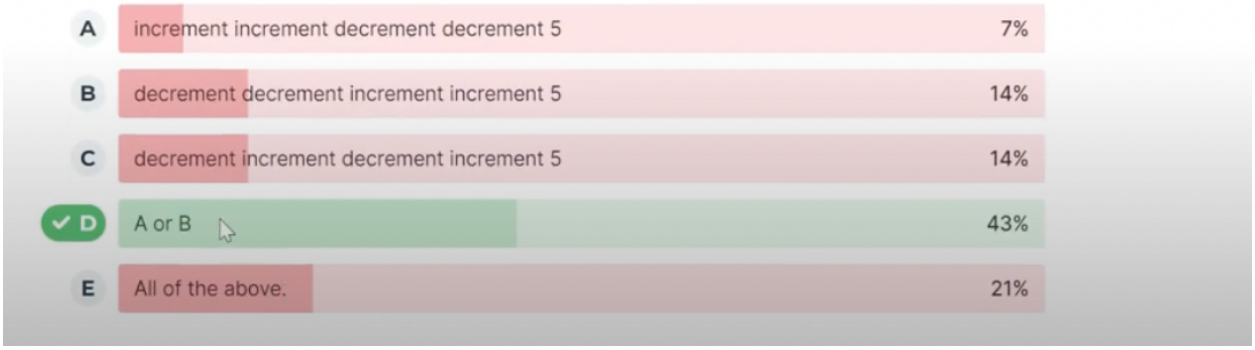
```
public synchronized void increment()throws InterruptedException
for(int i = 0; i < 2; i++) {
Random rand = new Random();
int num = rand.nextInt(2);
Thread.sleep(num*1000);
System.out.println("increment");
}
int temp = count;
Random rand = new Random();
int num = rand.nextInt(5);
Thread.sleep(num*1000);
temp++;
count = temp;
}
```

```
public synchronized void decrement() throws
InterruptedException{
for(int i = 0; i < 2; i++) {
Random rand = new Random();
int num = rand.nextInt(2);
Thread.sleep(num*1000);
System.out.println("decrement");
}
int temp = count;
Random rand = new Random();
int num = rand.nextInt(5);
Thread.sleep(num*1000);
temp--;
count = temp;
}
public int getCount() {
return count;
}
```

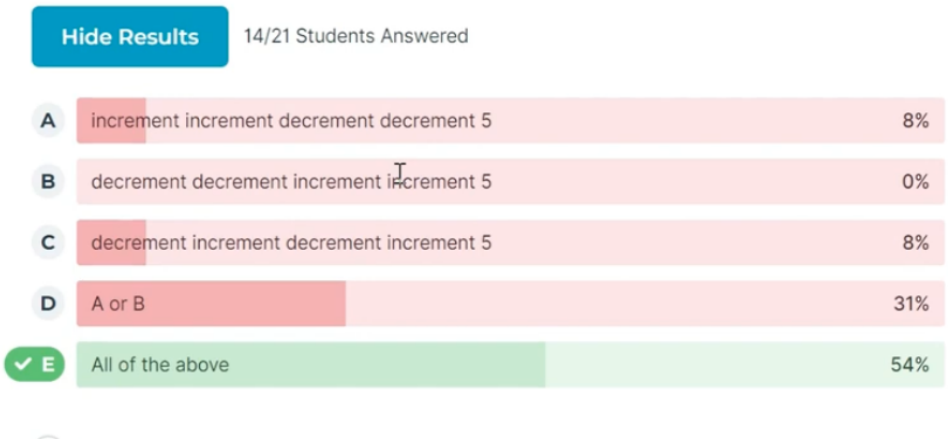
```
import java.util.concurrent.Semaphore;

public class Main {
public static void main (String[] args) throws InterruptedException {
Counter counter = new Counter(5);
Incrementer incThread = new Incrementer(counter);
Decrementer decThread = new Decrementer(counter);
incThread.start();
decThread.start();
incThread.join();
decThread.join();
System.out.println(counter.getCount());
}
}
```

What is(are) the possible output?

- there no way that increment and decrement will print overlap (synchronzied)



14. No synchronized in funciton increment and decrement



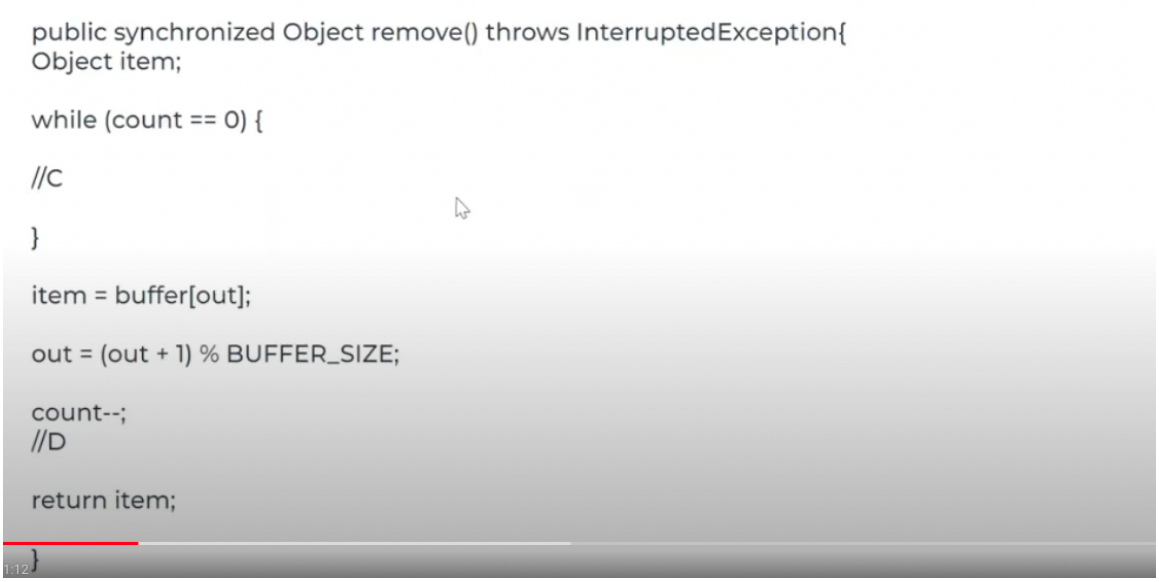
15. - 18

15. From the following BoundedBuffer class;

```
public class BoundedBuffer
{

private static final int BUFFER_SIZE = 5;

private int count; // number of items in the buffer

private int in; // points to the next free position in the buffer

private int out; // points to the next full position in the buffer

private Object[] buffer;

public BoundedBuffer()
```

```
public BoundedBuffer()
{
//
rbuffer is initially empty
count = 0;


in = 0;

out = 0;

buffer = new Object[BUFFER_SIZE];

}
//producer calls this method
```

```
public synchronized void insert(Object item) throws
InterruptedException {

while (count == BUFFER_SIZE) {

//A
}
buffer[in] = item;


in = (in + 1) % BUFFER_SIZE;

count++;
//B

}
// consumer calls this method
```



- A → wait()
  - do you use yield in synchronized method
- B → notify, notifyAll()
  - after finish notify
  - notifyAll → released everythread on wait set
- C → wait()
- D →notify, notifyAll()

Lab8

Operating Systems
Lab 8

1. Write a C program using the pthread's semaphore to implement the Bounded Buffer problem. You must implement pseudo-code in slide 6.22 as the "insertbuffer()" function and this function will be executed by a producer thread created by the main program. However, instead of using a forever loop, implement the "insertbuffer()" to run only forty rounds. The function needs to show the work of the producer thread as is shown in the example output. The pseudo-code in slide 6.23 needs to be implemented as the “readbuffer()” function where the consumer thread executes this function, the forever loop needs to be changed to run only forty rounds. The function needs to show the work of the producer thread as is shown in the example output. The main program initializes all semaphores, creates producer and consumer threads, waits for all threads to finish, and destroys all semaphores. The example output of the program is as follows:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
int count = 0;
int buffer[BUFFER_SIZE];
sem_t mutex;
sem_t empty;
sem_t full;
void *insertbuffer(void *param);
void *readbuffer(void *param);
int main() {
    pthread_t consumer;
    pthread_t producer;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    sem_init(&mutex,0,1);
```

```
    sem_init(&empty,0,5);
    sem_init(&full,0,0);
    pthread_create(&consumer, &attr, readbuffer, NULL);
    pthread_create(&producer, &attr, insertbuffer, NULL);
    pthread_join(consumer,NULL);
    pthread_join(producer,NULL);
    sem_destroy(&full);
    sem_destroy(&empty);
    sem_destroy(&mutex);
    return 0;
}
void *insertbuffer(void *param) {
    int i = 0;
    int in= 0;
    for(i = 0; i < 40;i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        if(count == BUFFER_SIZE) {
            printf("Producer Entered %d Buffer Full\n", i);
        }
        else {
            printf("Producer Entered %d Buffer size = %d\n", i,
        }
        sem_post(&mutex);
        sem_post(&full);
    }
}
void *readbuffer(void *param){
    int i =0;
    int out = 0;
    int item;
    for(i = 0; i < 40; i++) {
        sem_wait(&full);
```

```
        sem_wait(&mutex);
        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        if (count == 0) {
            printf("Consumer consumed %d Buffer Empty\n", item)
        }
        else {
            printf("Concumer consumed %d buffer size = %d\n", it
        }
        sem_post(&mutex);
        sem_post(&empty);
    }
}
```

Lab 9

Operating Systems
Lab 9

1. From the BoundedBuffer class attached to this exercise. Complete the class by initializing the constructor of the class to initialize all variables. Complete the "insert" method that will be called by a producer thread to insert data into the bounded buffer. This method also displays the work of the producer thread as is shown by the example output (you may use the code from slide 6.43 as the starting point). Complete the "remove" method that will be called by the consumer thread to read data from the bounded buffer. This method also displays the work of the consumer thread as is shown by the example output (you may use the code from slide 6.44 as the starting point). Write a Producer class as a Thread class that inserts 20 items into the buffer (one at a time, and when reaches the last slot of the buffer then goes back to insert the data to the first slot. Write a Consumer class as a Thread class that reads 20 items from the buffer (one at a time, and when reaches the last slot of the buffer then goes back to read the data from the first slot. Write a main program to create a BoundedBuffer object, a Producer object, a Consumer object by sharing the Bounded buffer object via the constructors of the Producer and Consumer classes. Start the Producer and Consumer threads. Wait for both threads to complete their tasks. The example output of the program is as follows:

```
public class Main
{
    public static void main(String args[]) throws InterruptedExc
        BoundedBuffer buffer = new BoundedBuffer();
        Thread producerThread = new Thread(new Producer(buffer))
        Thread consumerThread = new Thread(new Consumer(buffer))
        producerThread.start();
        consumerThread.start();
        producerThread.join();
        consumerThread.join();
    }
}
import java.util.*;

public class Producer implements Runnable
{
    private int number;
   private  BoundedBuffer buffer;
   public Producer(BoundedBuffer b) {
      buffer = b;
      number = 0;
   }
   public void run()
   {
         for(int i = 1; i <= 20; i++) {
        //while (true) {
            Random rand = new Random();
            try {
                        Thread.sleep(rand.nextInt(3000));
              }
              catch(InterruptedException e) {
              }
         // produce an item & enter it into the buffer

               //System.out.println("Producer produced " + numb
```

```
            buffer.insert(number++);
         }
   }


}
import java.util.*;

public class Consumer implements Runnable
{
   private  BoundedBuffer buffer;
   public Consumer(BoundedBuffer b) {
      buffer = b;
   }

   public void run()
   {
       int number  = 0;
      for(int i = 1; i <= 20; i++) {
      //while (true) {
         Random rand = new Random();
         try {
               Thread.sleep(rand.nextInt(3000));
           }
          catch(InterruptedException e) {
           }
         // consume an item from the buffer
         //System.out.println("Consumer wants to consume.");
         number = (Integer) buffer.remove();
         //System.out.println("Consumer consumes " + number);
        }
   }


}
public class BoundedBuffer
```

```
{
   private static final int   BUFFER_SIZE = 5;
   private int count; // number of items in the buffer
   private int in;   // points to the next free position in the
   private int out;  // points to the next full position in the
   private Object[] buffer;
   public BoundedBuffer()
   {
      // buffer is initially empty
      count = 0;
      in = 0;
      out = 0;
      buffer = new Object[BUFFER_SIZE];
   }
   //producer calls this method
   public synchronized void insert(Object item) {
      while (count == BUFFER_SIZE) {
         try {
            wait();
         }
         catch(InterruptedException e) {
         }
      }
        buffer[in] = item;
      in = (in + 1) % BUFFER_SIZE;
        count++;
        if (count == BUFFER_SIZE)
            System.out.println("Producer Entered " + item +
            " Buffer FULL");
        else
            System.out.println("Producer Entered " + item +
            " Buffer Size = " +  count);
     notify();
   }
   // consumer calls this method
   public synchronized Object remove() {
```

```
    Object item;
        while (count == 0) {
         try {
            wait();
         }
         catch(InterruptedException e) {
         }
      }
        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
      count--;
        if (count == 0)
           System.out.println("Consumer Consumed " + item +
         " Buffer EMPTY");
        else
            System.out.println("Consumer Consumed " + item +
            " Buffer Size = " + count);
      notify();
      return item;
   }
}
```

Lab 10

Operating Systems
Lab 10

1. Use the same code for the main program, producer thread and consumer thread from the Lab 9. But for this lab you need to complete the BoundedBuffer class using Java Semaphore. You can follow the algorithm from slides 6.21-6.23. You need to add the messages the show the work of each thread like you did in the Lab 9. The example output of the program is as follows:

```
public class Main
{
```

```
     public static void main(String args[]) throws InterruptedExc
         BoundedBuffer buffer = new BoundedBuffer();
         Thread producerThread = new Thread(new Producer(buffer))
         Thread consumerThread = new Thread(new Consumer(buffer))
         producerThread.start();
         consumerThread.start();
         producerThread.join();
         consumerThread.join();
     }
}
import java.util.concurrent.Semaphore;
public class BoundedBuffer
{
   private static final int   BUFFER_SIZE = 5;

   private int count; // number of items in the buffer
   private int in;   // points to the next free position in the
   private int out;  // points to the next full position in the
   private Object[] buffer;
   Semaphore mutex, full, empty;


   public BoundedBuffer()
   {
      // buffer is initially empty
      count = 0;
      in = 0;
      out = 0;
      buffer = new Object[BUFFER_SIZE];
      mutex = new Semaphore(1);
      full = new Semaphore(0);
      empty = new Semaphore(BUFFER_SIZE);
   }
   //producer calls this method
   public void insert(Object item) {
      try {
```

```
         empty.acquire();
         mutex.acquire();
      }
      catch(InterruptedException e) {
      }
        buffer[in] = item;
      in = (in + 1) % BUFFER_SIZE;
        count++;
        if (count == BUFFER_SIZE)
            System.out.println("Producer Entered " + item +
            " Buffer FULL");
        else
            System.out.println("Producer Entered " + item +
            " Buffer Size = " +  count);
    mutex.release();
    full.release();
  }

  // consumer calls this method
  public Object remove() {
   Object item;
     try {
          full.acquire();
         mutex.acquire();
     }
     catch(InterruptedException e) {
     }
       item = buffer[out];
       out = (out + 1) % BUFFER_SIZE;
     count--;
       if (count == 0)
          System.out.println("Consumer Consumed " + item +
         " Buffer EMPTY");
       else
           System.out.println("Consumer Consumed " + item +
           " Buffer Size = " + count);
```

```
      mutex.release();
      empty.release();
      return item;
   }
}
import java.util.*;

public class Producer implements Runnable
{
    private int number;
   private  BoundedBuffer buffer;
   public Producer(BoundedBuffer b) {
      buffer = b;
      number = 0;
   }
   public void run()
   {
         for(int i = 1; i <= 20; i++) {
        //while (true) {
            Random rand = new Random();
            try {
                        Thread.sleep(rand.nextInt(3000));
              }
              catch(InterruptedException e) {
              }
         // produce an item & enter it into the buffer

                //System.out.println("Producer produced " + numb
            buffer.insert(number++);
         }
   }


}
import java.util.*;
```

```
public class Consumer implements Runnable
{
   private  BoundedBuffer buffer;
   public Consumer(BoundedBuffer b) {
      buffer = b;
   }

   public void run()
   {
       int number  = 0;
      for(int i = 1; i <= 20; i++) {
      //while (true) {
         Random rand = new Random();
         try {
               Thread.sleep(rand.nextInt(3000));
           }
          catch(InterruptedException e) {
           }
         // consume an item from the buffer
         //System.out.println("Consumer wants to consume.");
         number = (Integer) buffer.remove();
         //System.out.println("Consumer consumes " + number);
        }
   }


}
```